

Implémentation et comparaison d'algorithmes d'apprentissage par renforcement sur cinq environnements

*Programmation dynamique, Monte Carlo, Temporal Difference Learning
et Planning*

2026-4A-IABD, (Deep) Reinforcement Learning P1

Groupe : [noms à compléter]

Juillet 2026

1. Introduction

Ce rapport présente le travail réalisé dans le cadre du projet de Reinforcement Learning. Le but du projet est d'implémenter les principaux algorithmes classiques de l'apprentissage par renforcement (programmation dynamique, Monte Carlo, Temporal Difference Learning et Planning) et de les tester sur plusieurs environnements pour comprendre dans quels cas chaque méthode fonctionne bien, et pourquoi.

Deux bibliothèques ont été développées séparément. La première regroupe les environnements (Line World, Grid World, Two Round Rock Paper Scissors et Monty Hall), qui suivent tous la même interface commune pour pouvoir être utilisés indifféremment par tous les algorithmes. La deuxième regroupe les algorithmes eux-mêmes. Cette séparation permet de tester n'importe quel algorithme sur n'importe quel environnement compatible sans avoir à réécrire de code spécifique à chaque fois.

La suite du rapport décrit d'abord les environnements et les algorithmes, puis présente les résultats obtenus lors de la comparaison de tous les algorithmes sur tous les environnements, et enfin une étude de l'impact de certains hyperparamètres (epsilon, gamma, nombre de pas de planification).

2. Les environnements

Chaque environnement expose la même interface : on peut le réinitialiser, jouer une action, savoir si la partie est terminée, connaître l'état courant et le score, et afficher l'état de façon lisible dans le terminal. Certains environnements exposent aussi le modèle complet des transitions (utile pour la programmation dynamique) et la possibilité de se placer directement dans un état donné (utile pour Monte Carlo ES).

2.1 Line World

L'agent se déplace sur une ligne de plusieurs cases. Il part du milieu et choisit à chaque tour d'aller à gauche ou à droite. Sortir par la gauche donne une récompense de -1, sortir par la droite donne +1. C'est l'environnement le plus simple du projet, il sert surtout de cas de base pour vérifier qu'un algorithme fonctionne correctement avant de le tester sur des environnements plus complexes.

2.2 Grid World

L'agent se déplace sur une grille (5x5 par défaut) en partant du coin en haut à gauche. Il doit atteindre une case objectif (récompense +1) en évitant une case piège (récompense -1). Une action qui ferait sortir l'agent de la grille le laisse simplement sur place, sans récompense. Cet environnement s'est révélé plus difficile que prévu pour certains algorithmes, comme expliqué dans la section des résultats.

2.3 Two Round Rock Paper Scissors

L'agent joue une partie de pierre-feuille-ciseaux en deux manches contre un adversaire particulier. Au premier round, l'adversaire joue au hasard. Au deuxième round, il est forcé

de rejouer exactement le coup que l'agent a joué au premier round. La bonne stratégie consiste donc à se souvenir de son propre coup pour le contrer au tour suivant, ce qui garantit une victoire au second round quel que soit le résultat du premier.

2.4 Monty Hall (niveaux 1 et 2)

Cet environnement reproduit le paradoxe de Monty Hall. Une porte gagnante est tirée au hasard parmi plusieurs portes. L'agent choisit une porte, puis le présentateur retire une porte perdante parmi celles non choisies. L'agent peut alors garder son choix ou en changer, et ainsi de suite jusqu'à ce qu'il ne reste plus que deux portes, moment où la porte choisie est ouverte. Le niveau 1 utilise 3 portes (une seule décision de garder ou changer), le niveau 2 en utilise 5 (trois décisions de suite). Les deux niveaux sont gérés par la même classe, seul le nombre de portes change.

Un point important à propos de cet environnement : l'identité précise d'une porte n'a pas de sens en soi, toutes les portes non choisies sont interchangeables. L'état montré aux algorithmes n'est donc pas "quelle porte est choisie" mais plutôt "combien de portes restent en jeu et quelle est la probabilité que le choix actuel soit gagnant". Cette probabilité suit une formule connue du paradoxe de Monty Hall : elle reste identique si l'agent garde son choix, et devient $(1 \text{ moins la probabilité actuelle}) \div (\text{nombre de portes restantes moins } 1)$ si l'agent change. Grâce à cette formule, l'environnement reste correct quel que soit le nombre de portes, sans avoir besoin d'écrire un cas particulier pour chaque niveau.

3. Les algorithmes

3.1 Programmation dynamique

Ces deux algorithmes supposent que l'on connaît déjà le modèle complet de l'environnement (toutes les probabilités de transition). Ils n'ont donc pas besoin de jouer d'épisode pour apprendre.

Value Iteration met à jour directement la valeur de chaque état avec la valeur de sa meilleure action possible, en répétant l'opération jusqu'à ce que la fonction de valeur ne bouge plus. Policy Iteration alterne deux étapes : évaluer complètement la politique actuelle, puis la rendre gloutonne par rapport à cette évaluation, jusqu'à ce que la politique ne change plus.

Un problème a été rencontré avec Policy Iteration sur Grid World : sa toute première politique est tirée au hasard action par action, et peut donc contenir des actions qui ramènent l'agent sur lui-même (un mur heurté sans arrêt). Avec un gamma proche de 1, évaluer exactement une telle politique jusqu'à convergence demanderait des millions d'itérations pour un gain quasiment nul. La solution retenue est de limiter volontairement le nombre d'itérations de l'étape d'évaluation (une évaluation partielle suffit déjà à orienter l'étape d'amélioration suivante dans la bonne direction), et d'ajouter une limite de sécurité sur la boucle globale au cas où la politique ne se stabiliserait jamais.

3.2 Méthodes Monte Carlo

Ces méthodes n'ont pas besoin de connaître le modèle de l'environnement, elles apprennent en jouant des épisodes complets et en observant les retours obtenus.

Monte Carlo ES (Exploring Starts) démarre chaque épisode sur un état et une action tirés au hasard, ce qui garantit que toutes les paires état-action sont explorées sans avoir besoin d'une politique de comportement aléatoire. C'est la méthode qui exige le plus de l'environnement : il doit être capable de se placer directement dans un état donné.

L'On-policy first visit MC control ne triche pas sur l'état de départ : chaque épisode part toujours du même point, et c'est la politique elle-même qui garantit l'exploration en étant epsilon-soft (elle joue presque toujours la meilleure action connue, mais garde une petite chance de jouer une action au hasard).

L'Off-policy MC control sépare complètement la politique qui joue (une politique de comportement uniforme, qui explore tout sans biais) de la politique que l'on cherche à améliorer (gloutonne par rapport à Q). Comme les deux diffèrent, chaque retour observé est repondéré par un ratio d'échantillonnage préférentiel pour rester une estimation correcte de la politique cible.

3.3 Temporal Difference Learning

Contrairement à Monte Carlo, ces méthodes n'attendent pas la fin d'un épisode complet pour mettre à jour leurs estimations : elles se mettent à jour après chaque pas, en se servant de la valeur estimée du pas suivant comme approximation du retour restant (le principe du bootstrap).

Sarsa est on-policy : l'action utilisée pour l'estimation du pas suivant est la même que celle qui sera réellement jouée ensuite. Q-Learning est off-policy : il se sert directement de la meilleure action possible depuis l'état suivant, qu'elle soit jouée ensuite ou non, ce qui lui permet d'apprendre la politique optimale même si le comportement réel reste exploratoire.

3.4 Planning

Dyna-Q combine l'apprentissage direct de Q-Learning avec un modèle de l'environnement appris au fil des épisodes. À chaque pas réellement joué, l'algorithme mémorise la transition observée, puis rejoue plusieurs transitions déjà connues tirées au hasard, comme si elles venaient d'être vécues, pour obtenir plusieurs mises à jour de Q à partir d'une seule interaction réelle avec l'environnement. L'intérêt de cette approche apparaît clairement dans la section 6.

3.5 Un problème commun trouvé pendant les tests

En construisant la comparaison complète décrite dans la section 5, un bug a été trouvé dans Sarsa, Q-Learning, Dyna-Q et Monte Carlo ES. Quand plusieurs actions ont exactement la même valeur estimée, la fonction `argmax` de numpy choisit toujours la première d'entre elles. Sur Grid World, avec un gamma proche de 1, une action qui ramène l'agent sur lui-même (un mur heurté) n'est presque jamais pénalisée par la mise à jour, puisque sa cible

(récompense plus valeur du même état) reste quasiment égale à sa valeur actuelle. Si cette action gagne le départage dès le début, rien ne l'empêche ensuite de continuer à le gagner, et l'agent peut rester bloqué à répéter la même action inutile pendant tout l'entraînement. La correction retenue a été de départager les égalités au hasard plutôt que de toujours prendre la première, ce qui casse ce blocage.

4. Méthodologie expérimentale

Un harness d'expérimentation a été construit pour comparer tous les algorithmes sur tous les environnements de façon uniforme. Pour chaque combinaison, l'algorithme est entraîné avec ses hyperparamètres, puis la politique obtenue est évaluée sur 200 épisodes joués sans apprentissage (action gloutonne à chaque pas), ce qui donne un score moyen directement comparable d'un algorithme à l'autre.

Les algorithmes de programmation dynamique sont peu sensibles au hasard une fois convergés, ils n'ont donc été lancés qu'une seule fois par environnement. Les méthodes model-free dépendent beaucoup de l'exploration, elles ont donc été relancées sur 5 graines aléatoires différentes par combinaison, ce qui permet de mesurer un taux de réussite plutôt qu'un seul résultat qui pourrait être une coïncidence favorable ou défavorable.

Un run est considéré réussi si son score moyen atteint au moins le score optimal réel de l'environnement moins une marge de 0,1. Le score optimal réel est calculé une fois pour chaque environnement avec Value Iteration plutôt que d'utiliser un seuil identique partout, ce qui est important pour Monty Hall : le score optimal y est de 2/3 pour 3 portes et de 4/5 pour 5 portes, pas de 1.

Les hyperparamètres utilisés pour la comparaison de base sont les valeurs par défaut de chaque algorithme, à une exception près : epsilon a été porté à 0,3 (au lieu de 0,1) pour Sarsa, Q-Learning, Dyna-Q et l'on-policy MC control, à cause du problème d'exploration décrit dans la section 3.5 et étudié plus en détail dans la section 6.

5. Résultats de la comparaison

Cette section présente les résultats obtenus pour chaque environnement. Le score optimal réel de chaque environnement, calculé avec Value Iteration, sert de référence pour juger si un algorithme a bien convergé.

5.1 Line World

Tous les algorithmes atteignent 100% de réussite sur cet environnement, ce qui est attendu : avec seulement 5 états et un chemin très court vers la récompense, aucune méthode n'a de difficulté particulière ici.

Score optimal réel de cet environnement : 1.000.

Algorithme	Score moyen	Écart-type	Taux de réussite	Temps moyen
------------	-------------	------------	------------------	-------------

				(s)
Policy Iteration	1.000	0.000	100 %	0.02
Value Iteration	1.000	0.000	100 %	0.00
Monte Carlo ES	1.000	0.000	100 %	0.25
On-policy first visit MC	1.000	0.000	100 %	0.52
Off-policy MC	1.000	0.000	100 %	0.32
Sarsa	1.000	0.000	100 %	0.20
Q-Learning	1.000	0.000	100 %	0.26
Dyna-Q	1.000	0.000	100 %	0.52

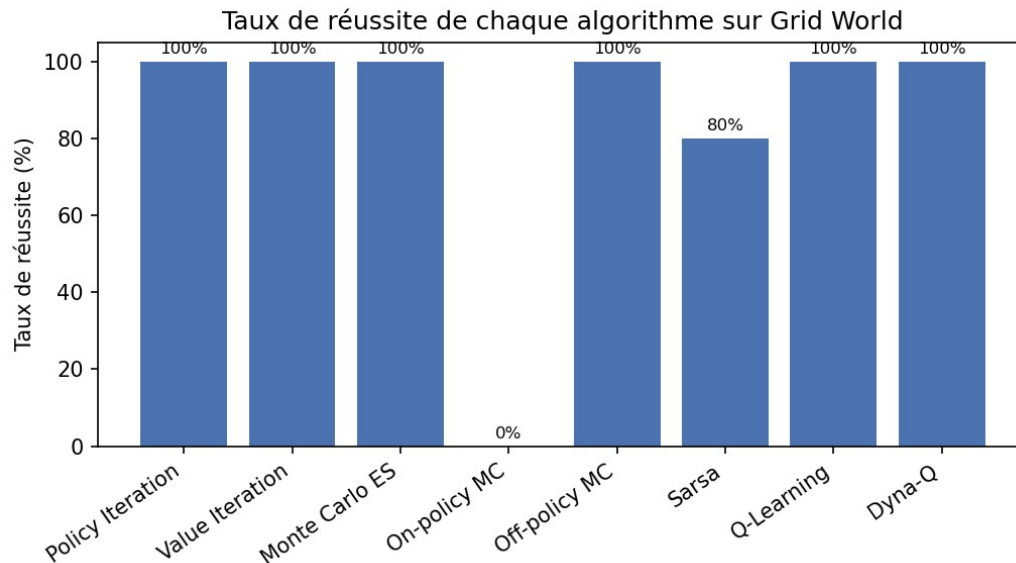
5.2 Grid World

C'est l'environnement où les différences entre algorithmes sont les plus visibles. La programmation dynamique, Monte Carlo ES, l'off-policy MC control, Q-Learning et Dyna-Q atteignent 100% de réussite. Sarsa réussit dans 80% des cas. L'on-policy first visit MC control échoue complètement (0%).

L'échec de l'on-policy MC control s'explique par sa nature même : cette méthode a besoin d'un épisode complet, du début jusqu'à un état terminal, pour faire la moindre mise à jour. Or sur Grid World, avec le problème de blocage sur les murs décrit en section 3.5, très peu d'épisodes atteignent effectivement l'objectif ou le piège pendant l'entraînement. Sarsa et Q-Learning, eux, apprennent un peu à chaque pas grâce au bootstrap, même si l'épisode ne se termine jamais, ce qui les rend plus robustes à ce problème.

Score optimal réel de cet environnement : 1.000.

Algorithme	Score moyen	Écart-type	Taux de réussite	Temps moyen (s)
Policy Iteration	1.000	0.000	100 %	1.30
Value Iteration	1.000	0.000	100 %	0.06
Monte Carlo ES	1.000	0.000	100 %	0.72
On-policy first visit MC	0.000	0.000	0 %	8.81
Off-policy MC	1.000	0.000	100 %	3.78
Sarsa	0.800	0.400	80 %	10.55
Q-Learning	1.000	0.000	100 %	2.01
Dyna-Q	1.000	0.000	100 %	3.14



5.3 Two Round Rock Paper Scissors

Tous les algorithmes obtiennent un score moyen proche de 1, ce qui correspond à la stratégie optimale (le premier round est neutre en espérance car l'adversaire y joue au hasard, le second round est gagné à coup sûr en contrant son propre coup du premier round). Les quelques taux de réussite en dessous de 100% viennent surtout du bruit d'évaluation : comme le premier round reste aléatoire, le score moyen sur 200 épisodes peut varier légèrement d'une graine à l'autre même avec une politique optimale.

Score optimal réel de cet environnement : 1.000.

Algorithme	Score moyen	Écart-type	Taux de réussite	Temps moyen (s)
Policy Iteration	1.015	0.000	100 %	0.00
Value Iteration	1.010	0.000	100 %	0.00
Monte Carlo ES	1.015	0.061	100 %	0.20
On-policy first visit MC	0.960	0.069	80 %	0.51
Off-policy MC	0.976	0.034	100 %	0.22
Sarsa	0.945	0.060	60 %	0.20
Q-Learning	0.975	0.061	80 %	0.19
Dyna-Q	1.015	0.029	100 %	0.32

5.4 Monty Hall, niveau 1 (3 portes)

Le score optimal réel de cet environnement est de $2/3$, le résultat classique du paradoxe de Monty Hall (toujours changer de porte double les chances de gagner par rapport à garder son choix initial). Tous les algorithmes s'en approchent bien, à l'exception de Dyna-Q qui

réussit dans 60% des cas seulement, avec un score moyen plus bas et un écart-type plus élevé que les autres méthodes.

Score optimal réel de cet environnement : 0.667.

Algorithme	Score moyen	Écart-type	Taux de réussite	Temps moyen (s)
Policy Iteration	0.645	0.000	100 %	0.00
Value Iteration	0.645	0.000	100 %	0.00
Monte Carlo ES	0.682	0.034	100 %	0.32
On-policy first visit MC	0.670	0.027	100 %	0.64
Off-policy MC	0.667	0.029	100 %	0.30
Sarsa	0.676	0.033	100 %	0.25
Q-Learning	0.645	0.019	100 %	0.28
Dyna-Q	0.544	0.193	60 %	0.45

5.5 Monty Hall, niveau 2 (5 portes)

Le score optimal réel passe à 4/5 avec 5 portes. Le même schéma se répète : la plupart des algorithmes s'en approchent bien, Sarsa (80%) et surtout Dyna-Q (60%) sont un peu moins fiables. Dyna-Q semble donc avoir plus de mal sur cet environnement que sur les autres, ce qui pourrait venir de son modèle appris : Dyna-Q suppose un environnement déterministe et ne retient que la dernière transition observée pour chaque paire état-action, ce qui colle mal avec la nature un peu particulière de l'état de Monty Hall (une probabilité plutôt qu'une position concrète).

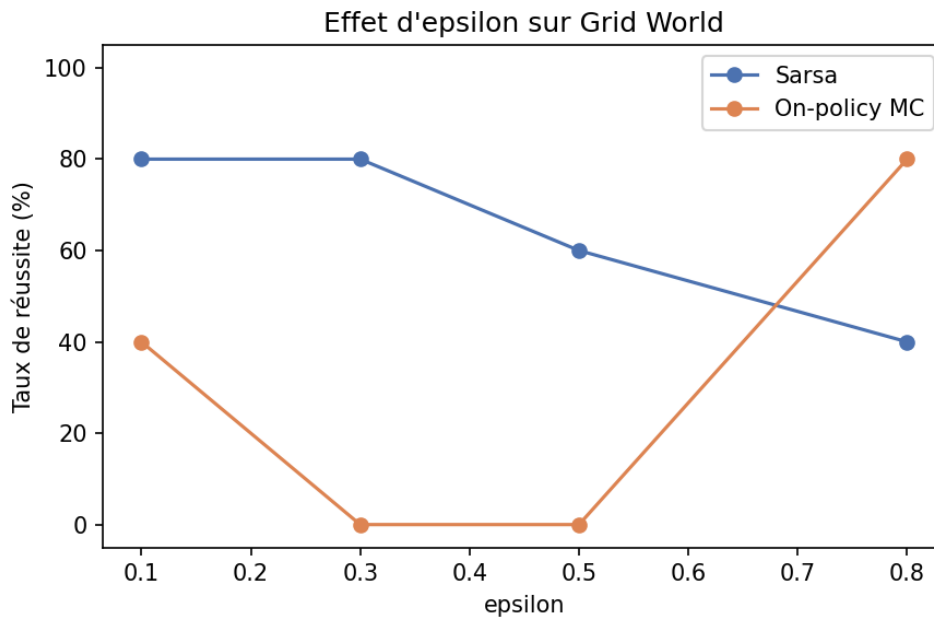
Score optimal réel de cet environnement : 0.800.

Algorithme	Score moyen	Écart-type	Taux de réussite	Temps moyen (s)
Policy Iteration	0.745	0.000	100 %	0.01
Value Iteration	0.735	0.000	100 %	0.00
Monte Carlo ES	0.783	0.028	100 %	0.37
On-policy first visit MC	0.794	0.028	100 %	1.21
Off-policy MC	0.786	0.021	100 %	0.57
Sarsa	0.744	0.094	80 %	0.55
Q-Learning	0.828	0.028	100 %	0.62
Dyna-Q	0.711	0.083	60 %	0.91

6. Étude des hyperparamètres

6.1 Epsilon sur Grid World

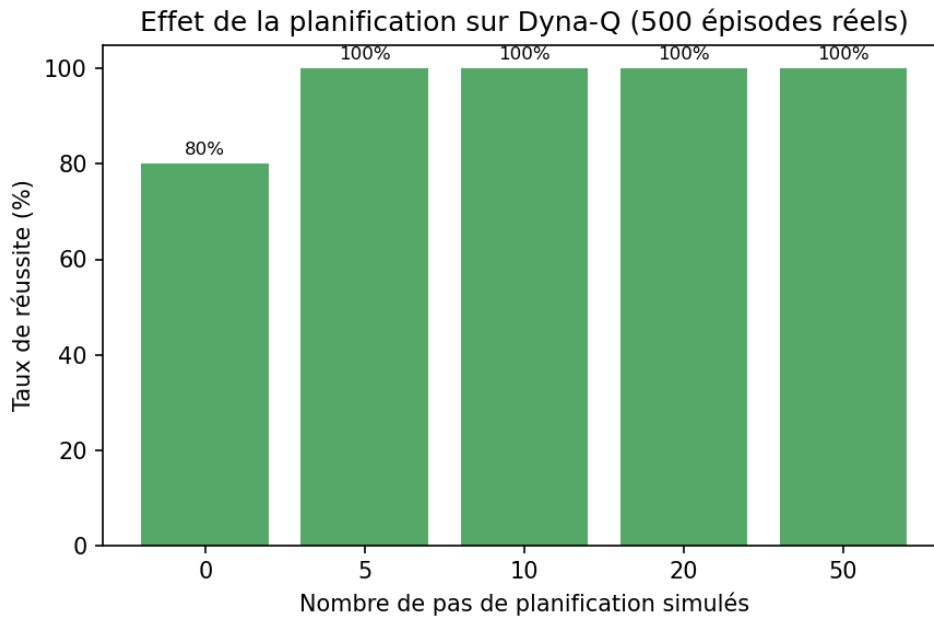
Cette étude reprend directement le problème de blocage trouvé en section 3.5, pour comprendre comment epsilon influence Sarsa et l'on-policy MC control sur Grid World.



Les deux courbes évoluent dans des sens opposés. Le taux de réussite de Sarsa baisse quand epsilon augmente au-delà de 0,3 (80% à 0,1 et 0,3, puis 60% à 0,5 et seulement 40% à 0,8) : un epsilon trop grand rend le comportement trop aléatoire, ce qui nuit à la qualité de la politique finale. L'on-policy MC control fait l'inverse : son taux de réussite est nul à 0,3 et 0,5, puis remonte à 80% pour epsilon égal à 0,8. Cette méthode a besoin qu'un épisode se termine ne serait-ce qu'une fois pour apprendre quoi que ce soit, elle a donc besoin de beaucoup plus d'exploration que Sarsa pour espérer atteindre un état terminal régulièrement.

6.2 Nombre de pas de planification sur Dyna-Q

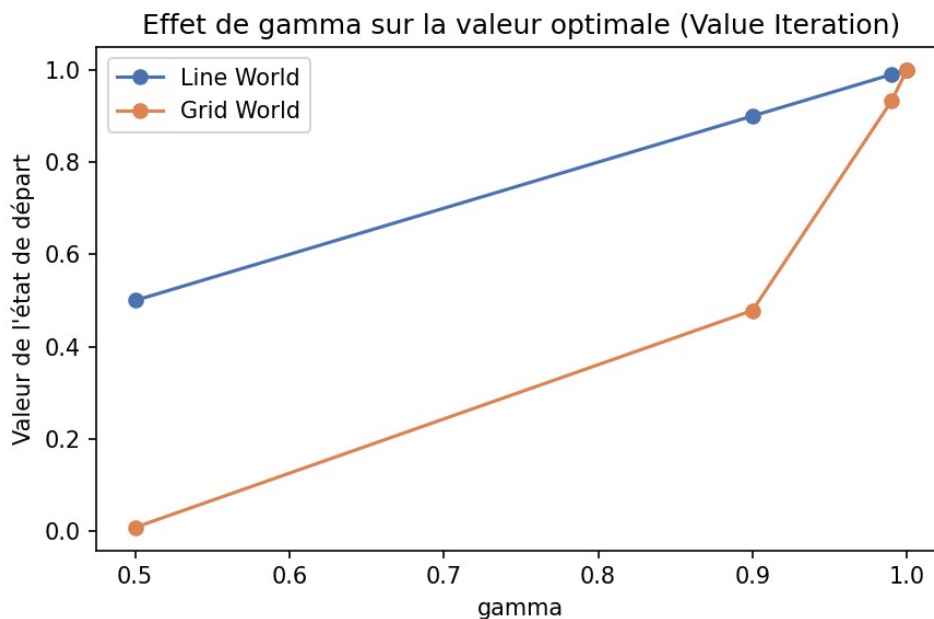
Cette étude compare Dyna-Q à lui-même avec un nombre croissant de pas de planification simulés par pas réel, à budget d'épisodes réels identique et volontairement petit (500 épisodes, pour bien montrer l'écart).



Avec 0 pas de planification (ce qui revient exactement à du Q-Learning), le taux de réussite est de 80%. Dès 5 pas de planification, le taux de réussite passe à 100%, et reste à 100% jusqu'à 50 pas. Cette expérience illustre bien l'intérêt de la planification : à nombre d'interactions réelles identique, rejouer des transitions déjà connues permet d'obtenir plus de mises à jour utiles de Q, et donc une convergence plus fiable.

6.3 Gamma sur Value Iteration

Cette étude regarde comment le facteur d'actualisation gamma influence la valeur optimale trouvée par Value Iteration, sur Line World et sur Grid World.



Sur Line World, la valeur suit presque directement gamma (0,5 donne une valeur de 0,5, 0,9 donne 0,9, etc.), car le chemin jusqu'à la récompense y est très court. Sur Grid World, l'effet est beaucoup plus marqué : avec gamma égal à 0,5, la valeur de l'état de départ tombe à 0,008, quasiment nulle, alors que le chemin optimal ne fait que 8 pas. Avec gamma égal à 0,9 la valeur remonte à 0,478, avec 0,99 à 0,932, et avec 0,999999 elle est proche de 1. Cette étude montre concrètement pourquoi un gamma proche de 1 est nécessaire dès que l'horizon de l'environnement s'allonge un peu : un gamma trop petit dévalue très fortement les récompenses qui demandent plusieurs pas pour être atteintes.

7. Intégration et résultats sur les environnements secrets

Les environnements secrets (0 à 3) sont fournis par le cours sous forme de bibliothèque compilée et d'un wrapper Python (`secret_envs/`), déjà disponibles avant même la fin du cours : seule l'interface graphique manque encore. Une classe adaptatrice les rend compatibles avec l'interface commune du projet, ce qui les rend utilisables par tous les algorithmes model-free (Sarsa, Q-Learning, Dyna-Q, Monte Carlo on-policy/off-policy) sans écrire de code spécifique à chacun.

Deux limites assumées : ces environnements n'exposent pas de moyen de se replacer directement dans un état donné, donc Monte Carlo ES ne s'y applique pas. Leurs espaces d'états (de 8192 à plus de 2 millions d'états) rendent aussi la programmation dynamique impraticable avec l'implémentation du projet, qui énumère explicitement toutes les combinaisons (état, action, état suivant, récompense) à chaque passage.

Environnement	Algorithme	Score moyen
Secret Env 0 (8192 états)	Dyna-Q	10.0
Secret Env 0 (8192 états)	On-policy first visit MC	10.0
Secret Env 1 (65536 états)	Dyna-Q	31.0
Secret Env 1 (65536 états)	Sarsa	30.0
Secret Env 2 (2 097 152 états)	Dyna-Q	-14.0 (meilleur score, mais négatif)
Secret Env 3 (65536 états)	(tous les algos testés)	Inutilisable : la bibliothèque plante

Secret Env 3 n'a pas pu être testé du tout : la bibliothèque compilée fournie plante systématiquement (message "Forbidden action") sur cet environnement, quel que soit l'algorithme testé (les 5 ont été essayés) et quelle que soit la graine aléatoire utilisée (3 tentatives par algorithme). Ce comportement n'a pas pu être reproduit de façon isolée pour en identifier la cause exacte : c'est une limite de la bibliothèque fournie, pas un problème dans le code du projet, qui gère cet échec proprement (chaque combinaison tourne dans un processus séparé, pour qu'un plantage n'empêche pas de tester les autres).

Sur les 3 environnements secrets exploitables, Dyna-Q obtient le meilleur score sur Secret Env 0 et Secret Env 1, ce qui confirme l'avantage de la planification déjà observé sur Grid

World (section 6.2). Sur Secret Env 2 (plus de 2 millions d'états), tous les algorithmes obtiennent un score négatif : avec seulement 1500 à 3000 itérations (budget réduit pour rester dans un temps raisonnable), la couverture de cet espace d'états est trop faible pour apprendre une bonne stratégie. C'est une limite honnête du travail réalisé, pas un échec caché : il faudrait beaucoup plus d'itérations, ou une méthode capable de généraliser entre états proches plutôt que d'apprendre chaque état indépendamment (ce que ne font pas les méthodes tabulaires utilisées ici), pour espérer un meilleur résultat sur un espace d'états aussi grand.

8. Quel algorithme choisir, et pourquoi

Sur les environnements les plus simples (Line World, Two Round RPS, Monty Hall), tous les algorithmes se valent à peu près, la question du choix ne se pose donc pas vraiment. Les différences intéressantes apparaissent sur Grid World, qui a un espace d'états plus grand et une récompense qui n'arrive qu'après plusieurs pas coordonnés.

Quand le modèle de l'environnement est connu à l'avance, la programmation dynamique reste le choix le plus sûr et le plus rapide : elle ne dépend d'aucune exploration et converge de façon fiable, sur les cinq environnements testés. Quand le modèle n'est pas connu, Q-Learning est la méthode la plus robuste dans l'ensemble. Dyna-Q apprend plus vite que Q-Learning à nombre d'épisodes réels égal sur Grid World grâce à sa planification (section 6.2), mais c'est aussi l'algorithme le moins fiable sur les deux niveaux de Monty Hall, sans doute à cause de son modèle appris qui suppose un environnement déterministe. Sarsa reste correct mais un peu moins fiable que Q-Learning sur Grid World. L'on-policy first visit MC control est clairement le moins adapté à Grid World dans sa configuration par défaut, à cause de son besoin d'épisodes complets. Monte Carlo ES et l'off-policy MC control restent fiables sur tous les environnements testés, mais Monte Carlo ES a besoin que l'environnement sache se replacer directement dans un état donné, ce qui n'est pas toujours possible.

En résumé, le choix d'un algorithme ne dépend pas seulement de l'environnement, mais aussi de ce que l'on sait sur lui à l'avance (le modèle complet ou seulement la possibilité de jouer des épisodes) et de la façon dont l'exploration est gérée par rapport à la difficulté réelle d'atteindre une récompense.

9. Limites et perspectives

L'interface graphique des environnements secrets n'était pas encore disponible au moment de la rédaction de ce rapport, seule leur interface en ligne de commande a été utilisée (section 7). Secret Env 3 n'a pas pu être testé du tout (bibliothèque fournie qui plante systématiquement), et Secret Env 2 n'a pas de stratégie satisfaisante à cause de son espace d'états bien trop grand (plus de 2 millions d'états) pour le budget d'itérations utilisé. L'algorithme optionnel Dyna-Q+ n'a pas non plus été implémenté, faute de temps disponible avant la date de rendu.

Le problème de blocage décrit en section 3.5 a été corrigé, mais Dyna-Q reste moins fiable que les autres méthodes sur Monty Hall (section 5.4 et 5.5). Une piste à creuser serait d'adapter le modèle appris par Dyna-Q pour mieux gérer les environnements dont l'état encode une information probabiliste plutôt qu'une position concrète.

10. Conclusion

Ce projet a permis d'implémenter huit algorithmes classiques de l'apprentissage par renforcement et cinq environnements partageant une interface commune, puis de les comparer de façon systématique. Les résultats confirment que la plupart des algorithmes fonctionnent bien sur des environnements simples, mais que des différences importantes apparaissent dès que l'environnement devient un peu plus grand ou que la récompense demande plusieurs actions coordonnées pour être atteinte. Le travail a aussi permis de découvrir et corriger un vrai problème d'exploration lié au départage des égalités dans le calcul de la meilleure action, ce qui montre l'intérêt de tester les algorithmes sur des environnements variés plutôt que sur un seul cas simple.